

Chapter 2

Relational Model



- ⇒ Relational Data Structure
- ⇒ Fundamental Relational-Algebra-Operations
- ⇒ Additional Relational-Algebra-Operations
- ⇒ Extended Relational-Algebra-Operations
- ⇒ Modification of the Database



Mathematics define a relation to be a subset of a Cartesian product of a list of ***Domains***:

$$D_1 \times D_2 \times \dots \times D_n$$

Formally, given sets $D_1, D_2, \dots, D_n$ a *relation* r is a subset of $D_1 \times D_2 \times \dots \times D_n$

Thus a relation is a set of n-tuples $(a_1, a_2, \dots, a_n)$ where $a_i \in D_i$

The difference here are :

assign attribute name to every domain;

finite set;



Example: if

customer-name = {Jones, Smith, Curry, Lindsay}

customer-street = {Main, North, Park}

customer-city = {Harrison, Rye, Pittsfield}

Then $r = \{$ (Jones, Main, Harrison),
 (Smith, North, Rye),
 (Curry, North, Rye),
 (Lindsay, Park, Pittsfield),...}

is a relation over $\text{customer-name} \times \text{customer-street} \times$
 customer-city



Relation Schema(模式)

- $A_1, A_2, \dots, A_n$ are *attributes*, each attribute of a relation has a name;

- The set of allowed values for each attribute is called the domain of the attribute, $D_1, D_2, \dots, D_n$:

$$\{\{A_1\} \rightarrow D_1, \{A_2\} \rightarrow D_2, \dots, \{A_n\} \rightarrow D_n\}$$

- $R = (A_1, A_2, \dots, A_n)$ is a *relation schema*

E.g. *Customer-schema* =
(*customer-name*, *customer-street*, *customer-city*)

- $r(R)$ is a *relation* on the *relation schema* R

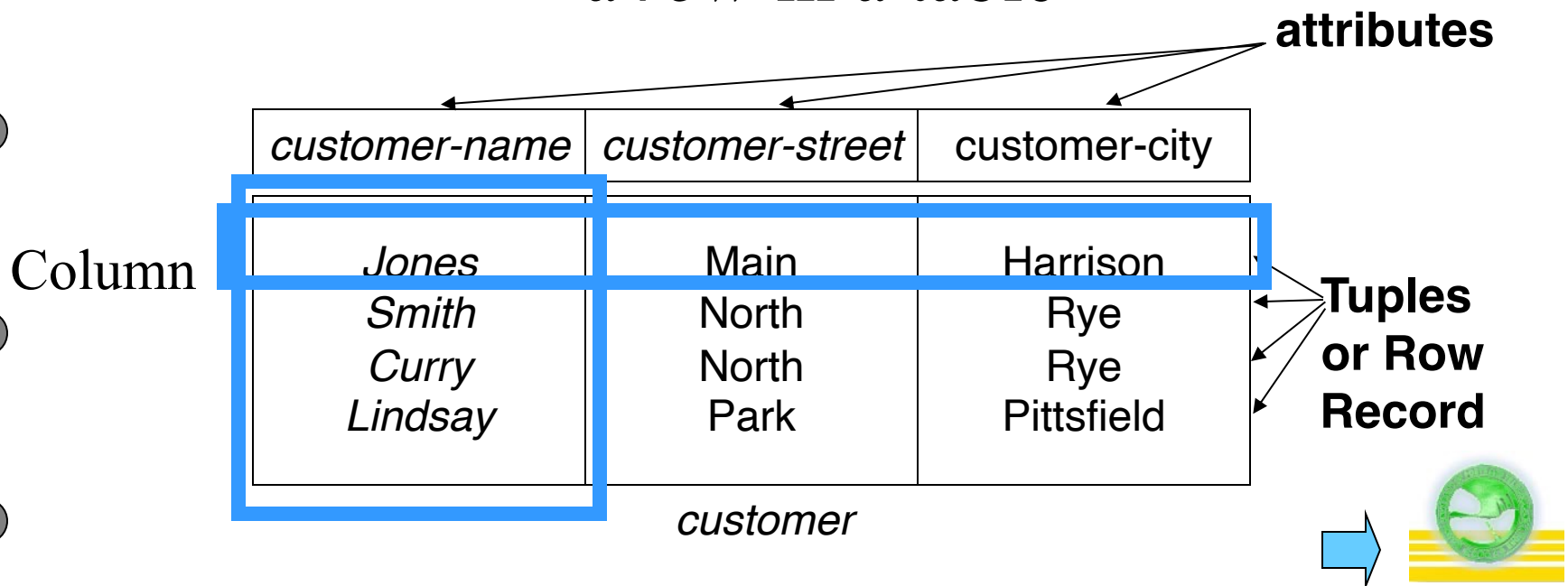
E.g. *customer* (*Customer-schema*)

- $R(A_1, A_2, \dots, A_n) / R(U)$



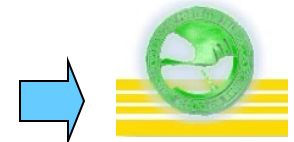
Relation Instance (实例)

- The current values (*relation instance*) of a relation are specified by a table
- An element t of r is a *tuple*, represented by a *row* in a table



NO	Name	Sex	Age	Class	Dept
7001	zhang	F	19	9901	4
7331	Li	M	19	9902	4
7004	Wang	F	20	0006	5

- Order of tuples is **irrelevant** (tuples may be stored in an arbitrary order); (行顺序无关)
- Order of attributes is **irrelevant**; (列顺序无关)
- For all relations , the domains of all attributes be **atomic**; (属性是原子的)
- Attributes names must **be different**; (属性名不同)
- Several attributes can have **the same domain**; (不同属性可来自相同域)
- Tuples are **not duplicate**. (行不重复)



Database Schema

- Database Schema:

- logical design of the database;

- Database instance:

- snapshot of the data in the database at a given instant in time;

- Relation schema;

- Relation instance;



$R(U)$ is a relation schema , $K \subseteq U$;

K is a **superkey**(超键) for R :

for any r , no two distinct tuples have the same values on K . That is ,if t_1 and t_2 are in r and $t_1 \neq t_2$, then $t_1[k] \neq t_2[k]$;

values for K are sufficient to **identify a unique tuple** of each possible relation $r(R)$ by “possible r ” ;

Example: $\{customer-name, customer-street\}$ and $\{customer-name\}$ are both superkeys of *Customer*,



- A *candidate key* (候选键) of an entity set is a **minimal superkey**

Customer-id is candidate key of *customer*

account-number is candidate key of *account*

- Although several candidate keys may exist, one of the candidate keys is selected to be the *primary key* (主键、关键字).

Example: student(no, id, name,sex ,age)

$\{no\}$ and $\{id\}$ are both candidate keys of student,
 $\{no\}$ can be selected as primary key;



Two Properties of Candidate Key:

Uniqueness (唯一性) :

No legal value of R ever contains two distinct tuples with the same value for K;

Irreducibility (最小性) :

No proper subset of K has the uniqueness property.



Foreign Key

For R1,R2, R1 includes the Primary key X of R2, X is called a *Foreign Key* (外键) referencing R2;

R1 is called the referencing relation of the foreign key dependence;

R2 is called the referenced relation of the foreign key ;

Example:

Employee(no,name,sex, age, deptno)

Dept(deptno, name , address)

上面哪张表的哪个或哪些属性是参照哪张表的外键?



问题:

- 1 外键的值可以为NULL吗?
- 2 外键的命名必须和他参照的主键名相同吗?
- 3 外键可以定义在同一张表上吗?



- The university is organized into departments. Each department is identified by a unique name (dept_name), is located in a particular building, and has a budget.
- Each department has a list of courses it offers. Each course has associated with it a courseid, title, deptname, and credits, and may also have associated prerequisites.
- Instructors are identified by their unique ID. Each instructor has name, associated department (dept_name), and salary.
- Students are identified by their unique ID. Each student has a name, an associated major department (dept_name), and tot_cred (total credit hours the student earned thus far).



- The university maintains a list of classrooms, specifying the name of the building, room_number and room capacity.
- The university maintains a list of all classes(sections) taught. Each section is identified by a course_id, sec_id, year, and semester and has associated with it a semester, year, building, room_number, and time_slot_id (the time slot when the class meets).
- The department has a list of teaching assignments specifying, for each instructor, the sections the instructor is teaching.
- The university has a list of all student course registrations, specifying, for each student, the courses and the associated sections that the student has taken (registered for).





Integrity Constraint of Primary Key:(实体完整性约束)

Each specified column of primary key is assumed to be NOT NULL.

Referential Integrity constraint: (参照完整性约束)

The FK-values in any tuple of relation r1 are either NULL or must appear as the PK-values of a tuple of relation r2.



Six basic operators

➤ Select(选择)

➤ Project (投影)

➤ Union (并)

➤ set difference (差)

➤ Cartesian product (笛卡尔乘积)

➤ Rename (更名)

The operators take one/two or more relations as inputs and give a new relation as a result.



Select Operation

- Notation: $\sigma_p(r)$ p is called the selection predicate

- Defined as:

$$\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$$

Where p is a formula in propositional calculus consisting of terms connected by : $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)

Each term is one of : G

$\langle \text{attribute} \rangle op \langle \text{attribute} \rangle$ or $\langle \text{constant} \rangle$

where op is one of: $=, \neq, >, \geq, <, \leq$

- Example of selection:

$\sigma_{dept_name = \text{"Physics"}}(instructor)$



Select Operation Example

- Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

- $\sigma_{A=B \wedge D > 5}(r)$

A	B	C	D
α	α	1	7
β	β	23	10



- Notation:

$$\Pi_{A_1, A_2, \dots, A_k} (r)$$

where A_1, A_2 are attribute names and r is a relation name.

- The result is defined as the relation of k columns obtained by erasing the columns that are not listed
- Duplicate rows removed from result, since relations are sets
- E.g. To eliminate the *dept_name* attribute of *instructor*
-

$$\Pi_{ID, name, salary} (instructor)$$



Project Operation Example

Relation r :

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

$\Pi_{A,C}(r)$

A	C
α	1
α	1
β	1
β	2

=

A	C
α	1
β	1
β	2



Union Operation

- Notation: $r \cup s$

- Defined as:

$$r \cup s = \{t \mid t \in r \text{ or } t \in s\}$$

- For $r \cup s$ to be valid.

1. r, s must have the same number of attributes

2. The attribute domains must be *compatible* (e.g., 2nd column of r deals with the same type of values as does the 2nd column of s)



E.g. to find all courses taught in the Fall 2009 semester, or in the Spring 2010 semester, or in both

$$\Pi_{course_id}(\sigma_{semester="Fall" \wedge year=2009}(section))$$

$\cup$

$$\Pi_{course_id}(\sigma_{semester="Spring" \wedge year=2010}(section))$$


Union Operation Example

Relations r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

$r \cup s$:

A	B
α	1
α	2
β	1
β	3



Set Difference Operation

- Notation $r - s$
- Defined as: find tuples that are in r but are not in s
$$r - s = \{t \mid t \in r \text{ and } t \notin s\}$$
- Set differences must be taken between *compatible* relations.
 - r and s must have the *same arity*
 - attribute domains of r and s must be compatible



Set Difference Operation Example *DataBase System Concepts*

Relations r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

$r - s$:

A	B
α	1
β	1



to find all courses taught in the Fall 2009 semester, but not in the Spring 2010 semester

$\Pi_{course_id}(\sigma_{semester="Fall" \wedge year=2009}(section))$

—

$\Pi_{course_id}(\sigma_{semester="Spring" \wedge year=2010}(section))$



- Notation $r \times s$: combine information from r and s

- Defined as:

$$r \times s = \{t \mid t \in r \text{ and } t \in s\}$$

- Assume that attributes of $r(R)$ and $s(S)$ are disjoint. (That is, $R \cap S = \emptyset$).

- If attributes of $r(R)$ and $s(S)$ are not disjoint, then renaming must be used.



Cartesian-Product Operation Example

Relations r, s :

A	B
α	1
β	2

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

$r \times s$:

A	B	C	D	E
α	1	α	10	a
α	1	β	19	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b



Rename Operation

- Allows us to name, and therefore to refer to, the results of relational-algebra expressions.

- Allows us to refer to a relation by more than one name. Example:

$\rho_x(E)$: returns the expression E under the name X

If a relational-algebra expression E has arity n , then

$$\rho_x(A1, A2, \dots, An)(E)$$

returns the result of expression E under the name X , and with the attributes renamed to $A1, A2, \dots, An$.



- A basic expression in the relational algebra consists of either one of the following:

A relation in the database

A constant relation

- Let E_1 and E_2 be relational-algebra expressions; the following are all relational-algebra expressions:

$$E_1 \cup E_2$$

$$E_1 - E_2$$

$$E_1 \times E_2$$

$$\sigma_p(E_1), P \text{ is a predicate on attributes in } E_1$$

$$\Pi_s(E_1), S \text{ is a list consisting of some of the attributes in } E_1$$

$$\rho_x(E_1), x \text{ is the new name for the result of } E_1$$



We define additional operations that do not add any power to the relational algebra, but that simplify common queries.

Set intersection (交)

(Natural) join (自然) 连接

Division (除法)

Assignment (赋值)



Set-Intersection Operation

- Notation: $r \cap s$

- Defined as:

$$r \cap s = \{ t \mid t \in r \text{ and } t \in s \}$$

Assume:

- r, s have the same number of attributes

- attributes of r and s are compatible

$$r \cap s = r - (r - s)$$



Set-Intersection Operation Example

Relation r, s:

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

$r \cap s$

A	B
α	2



▪ Notation: $\mathbf{r \bowtie_{A \theta B} s}$

$$\mathbf{r \bowtie_{A \theta B} s} = \{t \, q \mid t \in r \text{ and } q \in s \text{ and } (t[A] \theta q[B])\}$$

$$= \sigma_{A \theta B} (r \times s)$$

$$\theta \in \{=, \neq, \geq, \leq, >, <\}$$



Join Operation Example

Relations r, s :

A	B
3	1
6	2

C	D	E
1	2	3
4	5	6
7	8	9
10	11	12

$r \bowtie_{A>D} s$

A	B	C	D	E
3	1	1	2	3
6	2	1	2	3
6	2	4	5	6



Natural-Join Operation

Notation: $r \bowtie s$

$$r \bowtie s = \Pi_{\text{Att}(r) \cup (\text{Att}(s) - \{B\})} \sigma_{r.B=s.B} (r \times s)$$

B are the same attributes of r and s;



Natural Join Operation – Example

Relations r, s:

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ

s

$r \bowtie s$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ



Notation: $r \div s$

Let r and s be relations on schemas R and S respectively where

$$R = (A_1, \dots, A_m, B_1, \dots, B_n)$$

$$S = (B_1, \dots, B_n)$$

The result of $r \div s$ is a relation on schema

$$R - S = (A_1, \dots, A_m)$$

$$r \div s = \{ t \mid t \in \Pi_{R-S}(r) \wedge \forall u \in s (tu \in r) \}$$

$$= \Pi_{R-S}(r) - \Pi_{R-S}((\Pi_{R-S}(r) \times s) - r)$$



Division Operation – Example

Relations r, s:

A	B	C	D
a	b	c	d
a	b	e	f
b	c	e	f
e	d	c	d
e	d	e	f
a	b	d	e

r

C	D
c	d
e	f

s

$r \div s$

A	B
a	b
e	d

$$\Pi_{R-S}(r) - \Pi_{R-S}((\Pi_{R-S}(r) \times s) - r)$$



Assignment Operation

- The assignment operation ($\leftarrow$) provides a convenient way to express complex queries, write query as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as a result of the query.
- Assignment must always be made to a temporary relation variable.

Example: Write $r \div s$ as

$$temp1 \leftarrow \Pi_{R-S}(r)$$

$$temp2 \leftarrow \Pi_{R-S}((temp1 \times s) - r)$$

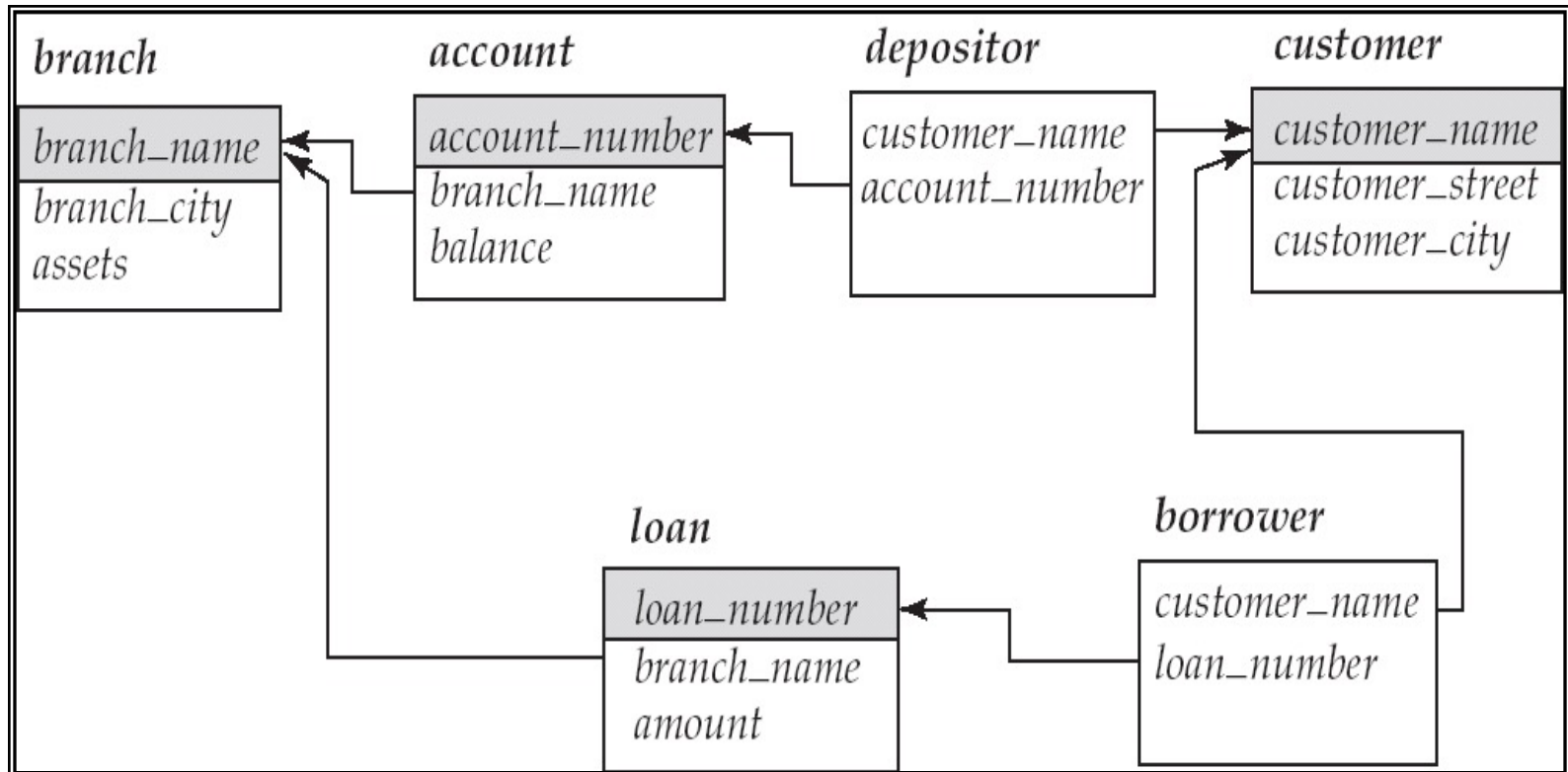
$$result = temp1 - temp2$$

The result to the right of the $\leftarrow$ is assigned to the relation variable on the left of the $\leftarrow$.

May use variable in subsequent expressions.



Banking Example



Banking Example

- branch (branch-name, branch-city, assets)
- customer (customer-name, customer-street, customer-only)
- account (account-number, branch-name, balance)
- loan (loan-number, branch-name, amount)
- depositor (customer-name, account-number)
- borrower (customer-name, loan-number)



Queries Example

- Find all loans of over \$1200

$$\sigma_{amount > 1200} (loan)$$

- Find the loan number for each loan of an amount greater than \$1200

$$\Pi_{loan-number} (\sigma_{amount > 1200} (loan))$$

- Find the names of all customers who have a loan, an account, or both, from the bank

$$\Pi_{customer-name} (borrower) \cup \Pi_{customer-name} (depositor)$$



Queries Example

- Find the names of all customers who have a loan and an account at bank.

$$\Pi_{customer-name}(borrower) \cap \Pi_{customer-name}(depositor)$$

- Find the names of all customers who have a loan at the Perryridge branch.

$$\Pi_{customer-name}(\sigma_{branch-name="Perryridge"}(\sigma_{borrower.loan-number = loan.loan-number}(borrower \times loan)))$$



Queries Example

- Find the names of all customers who have a loan at the Perryridge branch but do not have an account at any branch of the bank.

$\Pi_{customer-name} (\sigma_{branch-name = "Perryridge"}$

$(\sigma_{borrower.loan-number = loan.loan-number} (borrower \times loan)))$

– $\Pi_{customer-name} (depositor)$



Queries Example

- Find the largest account balance

$\Pi_{balance}(account) - \Pi_{account.balance}$

$(\sigma_{account.balance < d.balance} (account \times \rho_d$
 $(account)))$



Queries Example

- Find all customers who have an account from at least the “Downtown” and the Uptown” branches.

Query 1

$$\Pi_{CN}(\sigma_{BN=\text{“Downtown”}}(depositor \bowtie account)) \cap$$

$$\Pi_{CN}(\sigma_{BN=\text{“Uptown”}}(depositor \bowtie account))$$

where *CN* denotes customer-name and *BN* denotes *branch-name*.

Query 2

$$\Pi_{customer-name, branch-name}(depositor \bowtie account)$$

$$\div \rho_{temp(branch-name)}(\{(\text{“Downtown”}), (\text{“Uptown”})\})$$



Queries Example

- Find all customers who have an account at all branches located in Brooklyn city.

$$\Pi_{customer-name, branch-name} (depositor \bowtie account) \\ \div \Pi_{branch-name} (\sigma_{branch-city = \text{"Brooklyn"}} (branch))$$



Example2

S (SNO, SName, Sex, Age, Class, Dept) ,

C (Cno, Cname, Dept) ,

SC (SNO, Cno, Grade)

(1) $\Pi_{\text{SNO, GRADE}}(\delta_{\text{CNO}=\text{'C2'}}(\text{SC}))$

(2) $\Pi_{\text{SNO, SNAME}}(\delta_{\text{CNO}=\text{'C2'}}(\text{S} \bowtie \text{SC}))$

(3) $\Pi_{\text{SNO, SNAME}}(\delta_{\text{CNAME}=\text{'DB'}}(\text{S} \bowtie \text{SC} \bowtie \text{C}))$

(4) $\Pi_{\text{SNO, SNAME}}(\text{S}) - \Pi_{\text{SNO, SNAME}}(\delta_{\text{CNO}=\text{'C2'}}(\text{S} \bowtie \text{SC}))$

(5) $\Pi_{\text{SNAME}}(\text{S} \bowtie (\Pi_{\text{SNO, CNO}}(\text{SC}) \div \Pi_{\text{CNO}}(\text{C})))$



- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join.
- Uses *null* values:



Relation *loan*

<i>loan-number</i>	<i>branch-name</i>	<i>amount</i>
L-170	Downtown	3000
L-230	Redwood	4000
L-260	Perryridge	1700

Relation *borrower*

<i>customer-name</i>	<i>loan-number</i>
Jones	L-170
Smith	L-230
Hayes	L-155

Right Outer Join $\text{loan} \bowtie \text{borrower}$

<i>loan-number</i>	<i>branch-name</i>	<i>amount</i>	<i>customer-name</i>
L-170	Downtown	3000	Jones
L-230	Redwood	4000	Smith
L-260	Perryridge	1700	<i>null</i>
L-155	<i>null</i>	<i>null</i>	Hayes



- It is possible for tuples to have a null value, denoted by *null*, for some of their attributes
- *null* signifies an unknown value or that a value does not exist.
- The result of any arithmetic expression involving *null* is *null*.
- Aggregate functions simply ignore null values (as in SQL)
- For duplicate elimination and grouping, null is treated like any other value, and two nulls are assumed to be the same (as in SQL)



- Comparisons with null values return the special truth value: *unknown*
- Three-valued logic using the truth value *unknown*:
 - OR: (*unknown* **or** *true*) = *true*,
(*unknown* **or** *false*) = *unknown*
(*unknown* **or** *unknown*) = *unknown*
 - AND: (*true* **and** *unknown*) = *unknown*,
(*false* **and** *unknown*) = *false*,
(*unknown* **and** *unknown*) = *unknown*
 - NOT: (**not** *unknown*) = *unknown*
- Result of select predicate is treated as *false* if it evaluates to *unknown*



- Select: selection predicate p in $\sigma_p(r)$
- Join: the tuples which have a null value in a common attributes do not match
- Projection, Union, intersection, difference:
if two tuples in the results are exactly the same, and both have nulls in the same fields, they are treated as duplicates



- Aggregate functions simply ignore null values
 - Is an arbitrary decision. Could have returned null as result instead.
 - We follow the semantics of SQL in its handling of null values
- For duplicate elimination and grouping, null is treated like any other value, and two nulls are assumed to be the same
 - Alternative: assume each null is different from each other
 - Both are arbitrary decisions, so we simply follow SQL



- Deletion
- Insertion
- Updating



- A delete request is expressed similarly to a query, except instead of displaying tuples to the user, the selected tuples are removed from the database.
- Can delete only whole tuples; cannot delete values on only particular attributes
- A deletion is expressed in relational algebra by:

$$r \leftarrow r - E$$

where r is a relation and E is a relational algebra query.

- Delete all account records in the Perryridge branch.

$$account \leftarrow account - \sigma_{branch-name = "Perryridge"}(account)$$



- To insert data into a relation, we either:
 - specify a tuple to be inserted
 - write a query whose result is a set of tuples to be inserted
- in relational algebra, an insertion is expressed by:

$$r \leftarrow r \cup E$$

where r is a relation and E is a relational algebra expression.

- The insertion of a single tuple is expressed by letting E be a constant relation containing one tuple.

➤ Insert information in the database specifying that Smith has \$1200 in account A-973 at the Perryridge branch.

$$account \leftarrow account \cup \{(\text{"Perryridge"}, A-973, 1200)\}$$

$$depositor \leftarrow depositor \cup \{(\text{"Smith"}, A-973)\}$$



- Provide as a gift for all loan customers in the Perryridge branch, a \$200 savings account. Let the loan number serve as the account number for the new savings account.

$r_1 \leftarrow (\sigma_{branch_name = "Perryridge"}(borrower \bowtie loan))$

$account \leftarrow account \cup \Pi_{loan_number, branch_name, 200}(r_1)$

$depositor \leftarrow depositor \cup \Pi_{customer_name, loan_number}(r_1)$



- A mechanism to change a value in a tuple without changing *all* values in the tuple
- Use the generalized projection operator to do this task

$$r \leftarrow \Pi_{F_1, F_2, \dots, F_i} (r)$$

- Each F_i is either the i th attribute of r , if the i th attribute is not updated, or, if the attribute is to be updated, F_i is an expression, involving only constants and the attributes of r , which gives the new value for the attribute

➤ Make interest payments by increasing all balances by 5 percent.

$$account \leftarrow \Pi_{account-number, branch-name, balance * 1.05} (account)$$



■ Pay all accounts with balances over \$10,000 6 percent interest and pay all others 5 percent

$$\begin{aligned} \text{account} \leftarrow & \Pi_{\text{account_number}, \text{branch_name}, \text{balance} * 1.06} (\sigma_{\text{BAL} > 10000}(\text{account})) \\ & \cup \Pi_{\text{account_number}, \text{branch_name}, \text{balance} * 1.05} (\sigma_{\text{BAL} \leq 10000}(\text{account})) \end{aligned}$$


Relational Data Model

Table, Record/Tuple/Row, Attribute, domain/Column

Key, Candidate Key, Primary Key, Foreign Key

Constraint Rules

Relational algebra

Basic operations;

Additional operations;

Extended Operations;

Tuple relational Calculus; Domain relational Calculus.

